Truffles specification and context system

Operating manual for piggy-api and piggy-fe. Last reviewed: 2026-08-01.

This Markdown file is canonical. Do not hand-edit the PDF companion; regenerate it from this source with yarn docs:manual in piggy-api.

Executive summary

Truffles is best described as specification-informed and drift-enforced, with generated architectural context. It is not fully specification-driven in the strict sense that every implementation is generated from one executable specification.

The system combines authored rules and decisions with deterministic indexes generated from the code. Local pre-push hooks regenerate those indexes and block a push if the generated files are not committed. GitHub Actions independently checks the pushed main commit without changing files.

Code edits do not immediately update documentation. Run yarn context:build while developing, or let the pre-push hook regenerate it and then commit the resulting changes. Some source-of-truth documents still require deliberate human or agent edits.

The documentation layers

Agent entry points

Each repository has an AGENTS.md. This is the first orientation document for an AI agent or a new engineer. It explains the product, commands, hard architectural rules, key directories, and where deeper documentation lives.

The .github/copilot-instructions.md file is a thin compatibility entry point. Scoped files under .github/instructions/ load rules for particular paths, such as controllers, Prisma, tests, tax logic, frontend components, hooks, and PWA behavior. Their importance is precision: an agent sees the relevant rules without loading the entire repository into context.

Authored specifications

The docs/adr/ directories record durable architectural decisions and their rationale. They answer why the system works a certain way. Glossaries and methodology documents define domain language and business rules. Deployment documentation describes operational topology.

These files are not generated. A behavior or architectural decision change must update the relevant ADR, rule, glossary, or methodology document in the same change set.

Backend generated context

The backend context/ directory is generated by yarn context:build and must never be hand-edited.

- openapi.json: OpenAPI contract generated from controller Swagger JSDoc. It powers API contract

inspection and the frontend contract mirror.

- api-surface.md: route inventory with methods, paths, authentication, validators, and source locations. It makes endpoint discovery inexpensive.
- data-model.md: Prisma models, fields, relations, migrations, and a Mermaid ERD. It is the compact map of persisted data.
- module-graph.json and module-graph.md: dependency graph and architecture-rule violations. They expose forbidden imports and cycles.
- symbol-index.json: exported TypeScript symbols, signatures, documentation, and source locations. It helps tools navigate directly to owning code.
- unused.json: Knip findings for potentially unused files, exports, and dependencies. It is informational and requires review before deletion.
- manifest.json: SHA-256 hashes and generator version. It allows deterministic drift checking.

Frontend generated context

The frontend context/ directory is also generated by yarn context:build.

- route-map.md: Next.js routes, client boundaries, and imported components.
- component-index.md: component props, hooks, and API hooks used.
- api-binding.md: React Query hook to API method, query key, and invalidation relationships.
- data-flow.md: readers and invalidators for each query key.
- module-graph.json and module-graph.md: frontend dependency graph and architectural violations.
- symbol-index.json, unused.json, and manifest.json: the same navigation, cleanup, and integrity roles as their backend counterparts.
- contract-drift.md: warning report comparing the vendored OpenAPI schemas with handwritten frontend interface names.

The frontend also contains src/lib/generated/api-types.ts. It is generated reference material, not an application dependency. ESLint prevents production code from importing it.

Cross-repository contract

The backend owns context/openapi.json. The frontend vendors it as contracts/openapi.json, with source metadata in contracts/openapi.meta.json. Run yarn contract:pull in piggy-fe after a backend API change, then run yarn context:build there.

Updating the vendored copy remains a reviewed developer action. Freshness verification is automatic: frontend pre-push runs yarn contract:check against the sibling backend, and frontend CI sparsely checks out piggy-api/context/openapi.json and compares it with the committed contract.

What is automatic

yarn context:build deterministically regenerates architectural context without using an LLM. yarn context:check regenerates into a temporary directory, compares hashes, reports drift, and leaves the working tree unchanged.

On push, Husky runs context:build and contract:check. If generated files changed or the backend contract differs, the push is blocked so the relevant artifacts can be reviewed and committed. CONTEXT_SKIP=1 and CONTRACT_SKIP=1 are emergency bypasses and should not be routine.

After a push to main, GitHub Actions installs dependencies and runs lint, tests, build, context:check, and the cross-repository contract check. The workflow can also be started manually with workflow_dispatch. This hosted check reports a failure after the commit has landed; the local pre-push hook is what prevents stale generated context or contracts during the normal workflow. Both repositories' workflows were verified successfully on GitHub-hosted Ubuntu on 2026-08-01, including frontend contract verification. Dependency caching is non-authoritative, and the backend deploy bundle has an explicit one-day artifact retention.

What remains manual

- Update route Swagger JSDoc whenever a backend endpoint or request/response shape changes. OpenAPI

generation can reproduce the JSDoc but cannot prove it describes runtime behavior completely.

- Update piggy-fe/src/lib/types.ts when shared data shapes change. Generated API types are

reference-only, and contract-drift.md is warning-only.

- Run yarn contract:pull in the frontend after backend API changes.
- Create and review Prisma migrations for schema changes.
- Update ADRs, business rules, glossary entries, and tests when behavior or decisions change.
- Review generated diffs. Generation establishes consistency, not correctness.

Recommended change workflow

Backend-only implementation change

1. Change code and tests.
2. If a route changes, update its Swagger JSDoc. If a schema changes, create a migration.
3. Run yarn context:build.
4. Review and commit source, tests, authored docs, and generated context/ changes together.
5. Run yarn lint, yarn test --run, yarn build, and yarn context:check.
6. Push to main; the local hook checks generated files before upload and hosted CI validates the resulting commit.

Frontend-only implementation change

1. Change code and tests while preserving the hook and API boundaries in AGENTS.md.
2. Run yarn context:build.
3. Review and commit source, tests, authored docs, generated context, and generated reference types together.
4. Run yarn lint, yarn test --run, yarn build, and yarn context:check.
5. Push to main and confirm the hosted CI run succeeds.

Cross-repository API change

1. Implement and validate piggy-api, including Swagger JSDoc and generated backend context.
2. Commit the backend change first.

3. In piggy-fe, run yarn contract:pull.
4. Update handwritten frontend types and application code as needed.
5. Run yarn context:build, inspect context/contract-drift.md, and validate the frontend.
6. Commit the frontend contract, generated references, source, tests, and context together.

Current limitations and follow-ups

The frontend has no accepted architecture-violation baseline; every encoded violation fails context:check. Components, pages, and helper hooks consume backend operations through the central React Query layer.

Because hosted checks run after a direct push has landed, they detect failures but cannot prevent them from reaching main; recovery requires a follow-up fix or revert. Local hooks can also be bypassed, so review the GitHub Actions result after each push. Generator output should be reviewed after upgrades to TypeScript parsing, dependency analysis, Prisma, Next.js, Swagger, or OpenAPI tooling.

Token-efficiency expectations

Future AI-assisted changes should usually consume fewer tokens for repository discovery. Agents can read compact route, model, component, API-binding, and symbol indexes before opening implementation files. Scoped instructions also reduce irrelevant context.

This is a directional benefit, not a guaranteed percentage reduction. Small obvious edits may see little difference, while unfamiliar or cross-cutting work should benefit more. Generators and hash checks themselves use no model tokens. Implementation reasoning, tests, business-rule updates, Swagger JSDoc, type mirroring, contract pulls, and review still require work and context.

The practical measure is reduced search and rediscovery, not universally cheaper changes.

Ownership rule

Treat generated files as evidence and indexes, authored specifications as intent, tests as behavioral proof, and source code as implementation. A complete change keeps all four aligned.